ECE718: Optimizing Matrix Multiplication with Numba and Triton

Yuying Lai
laiy24@mcmaster.ca
400268588

The implementations of some code contained within this report were refined using the Claude AI platform. The grammar, wording, and format of this report were refined using the Gemini AI platform.

0. Table of Content

- 1 Introduction
  - 1.1 Background
  - 1.2 Objectives
  - 1.3 Scope of Techniques
- 2 Experimental Setup
  - 2.1 Hardware Specifications
  - 2.2 Software Environment
  - 2.3 Baseline Implementation
  - 2.4 Measurement Method
- 3 Numba Implementations and Results
  - 3.1 Implementation of Matrix Transposition Optimization in Numba
  - 3.2 Implementation of CPU Parallelism
  - 3.3 Implementation of Loop Tiling in Numba
  - 3.4 Implementation of Loop Interchange in Numba
  - 3.5 Implementation of Loop Unrolling in Numba
  - 3.6 Combining All Numba Optimizations
  - 3.7 Performance Analysis and Discussion
- 4 Triton Implementations and Results
  - 4.1 Implementation of Loop Tiling in Triton
  - 4.2 Implementation of 2D Grid Strategy
  - 4.3 Implementation of L2 Swizzling
  - 4.4 Implementation of Persistent Kernels
  - 4.5 Autotune in Triton
  - 4.6 Performance Analysis and Discussion
- 5 Comparative Analysis
- 6 Conclusion and Future Work
  - 6.1 Summary of Findings
  - 6.2 Limitations
  - 6.3 Recommendations for Further Research
- 7 Appendix
  - 7.1 Full Code Listings
- 8 Reference

1. Introduction

1.1 Background

Matrix multiplication (GEMM - General Matrix-Matrix Multiplication) constitutes a cornerstone operation in various computational fields, notably scientific computing, machine learning, and data analytics. The performance of this operation critically influences the overall efficiency of large-scale applications. While Python environments, utilizing libraries such as **NumPy (Harris et al., 2020) and PyTorch (Paszke et al., 2019)**, provide high-level abstractions for matrix operations, the underlying execution relies on highly optimized low-level implementations, typically written in C/C++ or CUDA.

However, optimal performance is not always guaranteed by these black-box libraries, particularly when dealing with specialized hardware or non-standard matrix dimensions. Consequently, this initiative seeks to investigate methodologies for enhancing matrix multiplication performance directly within the Python ecosystem through the application of Just-In-Time (JIT) compilation techniques. Specifically, we will leverage the **Numba framework (Lam et al., 2015)** for optimizing CPU performance and the **Triton framework (Tillet et al., 2019)** for optimizing GPU performance.

1.2 Objectives

The central objective of this research is to systematically investigate and quantitatively evaluate the performance improvements achievable for standard matrix multiplication (C = A * B) by implementing various optimization strategies using the Numba and Triton frameworks.

The specific goals are defined as follows:

- Establish baseline performance metrics for matrix multiplication using conventional high-level libraries (e.g., Naive Python/NumPy/CuPy).
- Implement and benchmark a spectrum of standard compiler and micro-architecture optimizations (e.g., tiling, loop unrolling, and parallelization) for CPU execution using the Numba framework.
- Implement and benchmark GPU-specific optimizations (e.g., memory coalescing, shared memory utilization, and kernel pipelining) using the Triton framework.
- Conduct a comprehensive comparative analysis between the peak performance achieved by the optimized Numba (CPU) implementation and the optimized Triton (GPU) implementation.

1.3 Scope of Techniques

This study will concentrate on the implementation and analysis of performance optimizations categorized by their target platform:

CPU Optimizations (Numba):

- Loop Transformations: Tiling, loop interchange, and loop unrolling.
- Memory Access Optimization: Matrix transposition to enhance cache locality.
- Parallelism: Exploiting multi-core CPU architectures using Numba's explicit parallel ranging (`prange`).

GPU Optimizations (Triton):

- Data Movement and Access: Utilizing memory coalescing and data prefetching.
- Kernel Execution Strategy: Implementing 2D grid partitioning and continuous kernel execution (persistent kernels).
- Computational Efficiency: Leveraging the GPU's inherent massive parallelism and optimizing data flow to utilize fast on-chip memory (shared memory), which is intrinsically managed through Triton's

tiling mechanisms.

2. Experimental Setup

2.1 Hardware Specifications

The experiments were conducted on a single machine to ensure consistent comparison between CPU and GPU optimizations.

CPU Specifications

The CPU utilized for the Numba-based optimizations is an Intel Core i7-11700. The key architectural specifications, particularly the cache hierarchy, are detailed below as reported by the `lscpu` utility.

Cache Level	Size (i7-11700)
L1d (Data)	384 KiB (8 instances)
L1i (Instruction)	256 KiB (8 instances)
L2 (Unified)	4 MiB (8 instances)
L3 (Shared)	16 MiB (1 instance)

GPU Specifications

The GPU utilized for the Triton-based optimizations is an NVIDIA GeForce GTX 1660. This midrange consumer GPU represents a typical target for entry-level machine learning and high-performance computing tasks, providing a realistic benchmark for the benefits of custom kernel development.

Component	Specification (GTX 1660)
Architecture	Turing
SM Count	22
VRAM	6 GB GDDR5

2.2 Software Environment

All software components were managed using the Conda environment manager to ensure a reproducible setup. The necessary libraries were installed and are utilized directly on the specified hardware.

Component	Version/Details
Operating System	Ubuntu 22.04.5 LTS
Python	3.11.14
Numba	0.62.1
Triton	3.1.0
NumPy	2.3.4
CuPy	13.6.0

2.3 Baseline Implementation

Three primary baselines are established to provide a comprehensive reference for the performance improvements achieved by Numba and Triton optimizations:

2.3.1 Naive Numba-Jitted Implementation

This baseline represents the simplest possible implementation of matrix multiplication, compiled by Numba without any explicit optimization directives (e.g., threading, tiling). It serves as the direct reference point for measuring the benefits of subsequent Numba-specific transformations.
The naive baseline is defined using a Numba Just-In-Time (JIT) compiled implementation (`@jit(nopython=True, cache=True)`) rather than a pure, uncompiled Python loop, because the latter would be prohibitively slow and unmeasurable in large scale under time constraint (e.g., taking around five minutes for a 2048x2048 matrix multiplication). By using the JIT-compiled version as the baseline, we ensure that the performance measurements are meaningful.

```
@jit(nopython=True, cache=True)
def numba_naive_mul(A, B, C):
    N = A.shape[0]
    for i in range(N):
        for j in range(N):
            tmp = 0.0
            for k in range(N):
                tmp += A[i, k] * B[k, j]
            C[i, j] = tmp
    return C
```

2.3.2 Highly Optimized Library Baselines (NumPy/CuPy)

These baselines use the highly optimized routines provided by standard numerical libraries, which typically rely on underlying BLAS (NumPy) or highly-tuned CUDA kernels (CuPy).

- NumPy (CPU Baseline): Utilizes the numpy.dot or numpy.matmul function, which links to highly optimized libraries such as **OpenBLAS (Zhang et al., 2012)** or Intel MKL specifically tuned for the CPU architecture.
- CuPy (GPU Baseline) **(Okuta et al., 2017)**: Utilizes the cupy.dot or cupy.matmul function, providing a near-optimal performance ceiling for the GPU using vendor-provided, tuned kernels. This serves as the target against which the custom Triton kernels will be measured.

2.4 Measurement Method

Numba (CPU) Measurement:

- Hardware Counters: `perf` utility monitors cycles, instructions, and various cache loads/misses (L1-dcache, L2, LLC) to analyze micro-architectural behavior.
- Wall-Clock Time: The Python `time` library measures actual execution time, avoiding the setup cost included by `perf`.

Triton (GPU) Measurement:

- Wall-Clock Time: CUDA Events (`torch.cuda.Event`) are used for accurate, asynchronous kernel timing on the GPU.

Protocol:

- Each benchmark runs 30 times to eliminate warm-up artifacts.
- Problem Size: 2048 x 2048 square matrices.
- Data Types: Numba uses NumPy datatype `np.float32`; Triton uses PyTorch datatype `torch.float32`.

3. Numba Implementations and Results

3.1 Implementation of Matrix Transposition Optimization in Numba

Transposing the matrix operand significantly improves performance in matrix multiplication by optimizing memory access patterns. This technique transforms a column-major access pattern into a row-major access pattern, which is much more efficient on modern CPUs that employ cache lines. By accessing data sequentially in memory after transposition, the CPU can maximize cache hits by utilizing spatial locality and minimize the latency associated with fetching data from main memory, leading to a substantial speedup across various implementation scenarios. To implement transpose. use `B_T = B.T.copy()`

The performance analysis reveals that matrix transposition yields considerable speedups across all tested implementation strategies. Compared to naive implementation, it saw a remarkable 2.94x speedup, while under parallelism transpose also brings a 4.72x speed up compared to naive parallel. Even the tiled approach, which is already optimized for cache usage, still benefited from transposition, achieving a 1.26x speedup. This consistent improvement across all scenarios underscores the importance of memory layout and cache optimization, confirming that transposing the matrix is a highly effective, fundamental optimization for high-performance matrix multiplication.

However, it must be noted that transposing and copying matrix B incurs additional memory load and store overhead during the setup phase. This approach is only advantageous when matrix B is frequently reused, such that the improvement in latency surpasses the initial setup overhead.

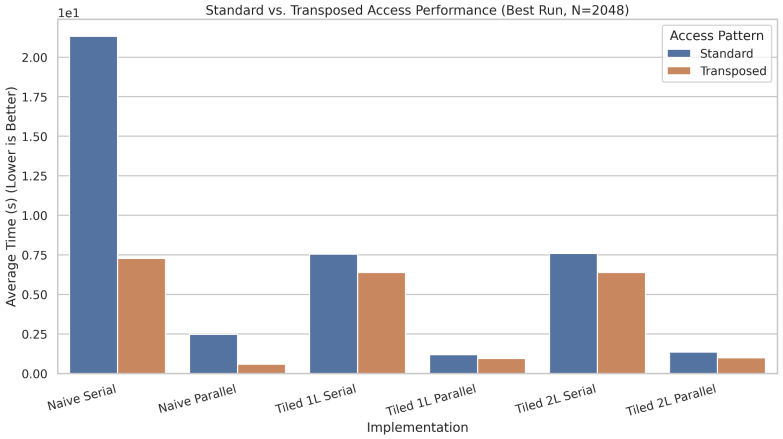


Fig1: Performance Impact of Matrix Transposition on average runtime of matrix multiplication

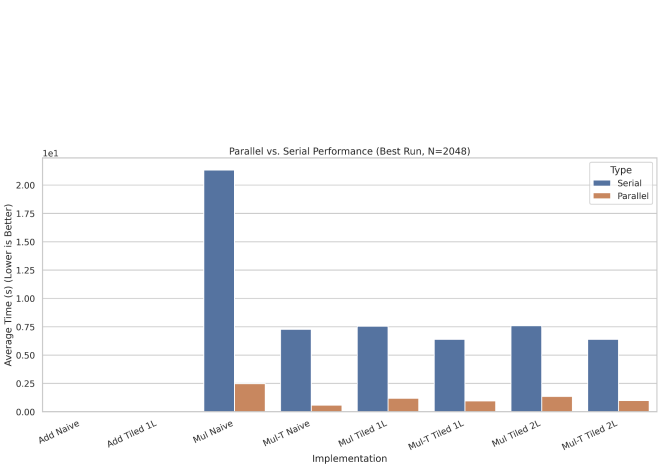
3.2 Implementation of CPU Parallelism

The implementation of NUMBA parallel computing significantly accelerated the matrix multiplication benchmarks. This technique leverages multiple processor cores to execute the computations concurrently, leading to substantial performance gains across all tested scenarios. The speedups observed demonstrate the effectiveness of parallelization in optimizing computationally intensive tasks. To use parallelism, use `parallel=True` in Numba jit decorator.

The performance gains were most pronounced under the naive and naive transpose benchmarks. Specifically, the naive implementation saw an impressive 8.64x speedup when parallelization was introduced. The naive transpose benchmark achieved an even greater acceleration, boasting a 12.5x speedup. While still significant, the tiled implementation exhibited a slightly lower average speedup of 7x, indicating that parallelism effectively complements various optimization strategies.

Further investigation into threading performance by setting number of threads in environmental variable `NUMBA_NUM_THREADS=$threads`, conducted on a system using the Intel Core i7-11700 processor (featuring 8 physical cores and 16 logical threads via Hyper-Threading), revealed a non-linear relationship between the

number of threads and execution speed. While increasing the thread count initially yielded dramatic improvements, the marginal benefit diminished as the number of threads grew higher. The most optimal performance was consistently achieved with 16 threads, suggesting a balance where overhead from managing more threads begins to counteract the parallelization benefits, aligning with the processor's maximum logical thread count.



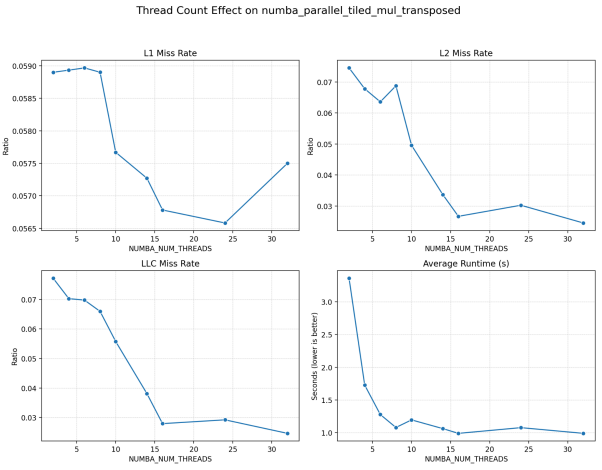


Fig2: Comparative Analysis of Serial vs. Parallel Execution. Figure 3: The Impact of Threading Count on Performance Across Cache Levels and Mean Runtime

3.3 Implementation of Loop Tiling in Numba

Tiling, also known as blocking, is a memory optimization technique that divides a large data structure (like a matrix for multiplication) into smaller, fixed-size blocks or tiles **(Wolf & Lam, 1991)**. The core idea is to improve performance by exploiting the principle of locality, specifically temporal and spatial locality, within the processor's memory hierarchy. By processing a small tile of data completely before moving to the next, tiling ensures that the actively used data remains in the faster, smaller cache levels (L1, L2, LLC). This dramatically reduces the number of costly accesses to the slower main memory (DRAM), which is the primary bottleneck in many compute-intensive applications like matrix operations.
Our implementation of Level 1 blocking:

```
@jit(nopython=True, cache=True)
def numba_tiled_mul(A, B, C, B1):
    N = A.shape[0]
    for ii in range(0, N, B1):
        for jj in range(0, N, B1):
            for kk in range(0, N, B1):
                for i in range(ii, min(ii + B1, N)):
                    for j in range(jj, min(jj + B1, N)):
                        tmp = 0.0
                        for k in range(kk, min(kk + B1, N)):
                            tmp += A[i, k] * B[k, j]
                        C[i, j] += tmp
    return C
```

The provided graph illustrates the effectiveness of tiling, showing a significant decrease in L2 and Last-Level Cache (LLC) misses, which directly translates to improved runtime. For instance, moving beyond the naive implementation, a block size of 64 yielded the greatest performance gain with a speed-up of 2.83x. Even when incorporating a B-transpose optimization, tiling still provided a measurable benefit, with a block size of 32 resulting in an additional 1.14x speed-up over the B-transpose baseline. These results confirm that selecting the optimal tile size is a critical factor in maximizing cache utilization and achieving substantial performance improvements.

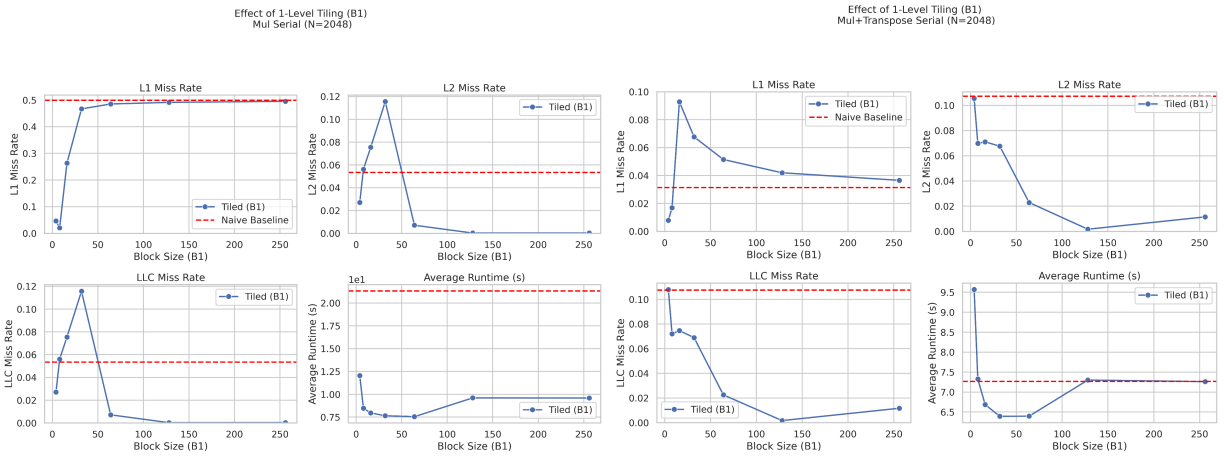


Fig 4: illustrates the influence of various loop tiling block sizes on performance across different cache levels and the mean runtime, depicted for both B (left) and B transpose (right) execution. The naive benchmark is indicated by the red dashed line (left), and the B transpose naive benchmark is similarly represented (right).

We also see that 1-level tiling benefits multicore performance when a transpose operation is not utilized, specifically with a 32 block size. However, blocking does not provide a performance advantage when multicore parallelism is combined with B transpose. This is perhaps because the transpose operation already significantly improves data locality by ensuring consecutive memory accesses, which limits the benefit that additional blocking can provide. Furthermore, maybe introducing multicore parallelism alongside the transpose can lead to cache contention and thrashing, where parallel threads compete for and evict each other's data from the cache, nullifying the locality improvements gained from both the transpose and blocking, and there may be extra scheduling overhead.
Please note that the cache miss rate and the actual runtime do not appear to correlate accurately in the graph, as the cache measurement utilizes 'perf' which encompasses the initial setup cost. We use the runtime measurement as the main comparing parameter and ‘perf’ just as a reference.

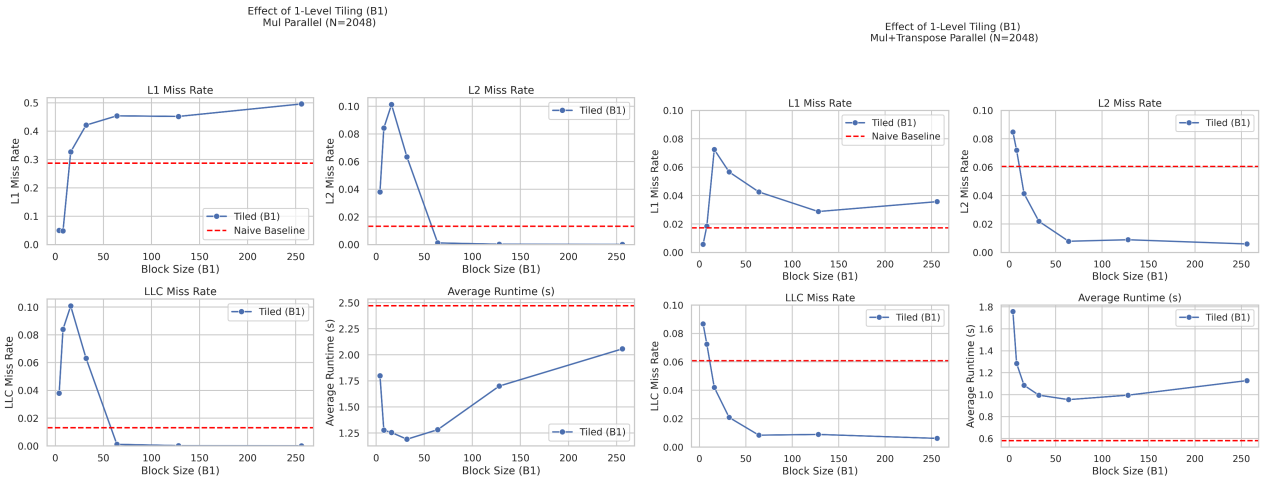


Fig 5: illustrates the influence of various loop tiling block sizes on performance across different cache levels and the mean runtime, depicted for both paralle (left) and B transpose+ parallel (right) execution. The naive parallel benchmark is indicated by the red dashed line (left), and the B transpose + parallel + naive benchmark is similarly represented (right).

We further analyze 2 layer tiling to see if we can further increase L1 or L2 locality. Two-layer tiling (also known as blocking) involves applying a tiling transformation twice, first to divide the entire iteration space into larger blocks (the first layer), and then applying a second tiling transformation to subdivide those large blocks

into smaller blocks (the second layer), which are typically sized to fit within a specific cache level (e.g., L1 cache). The goal is to maximize data reuse within the smaller blocks, thereby improving cache locality. However, simply implementing 2 layer tiling does not always yield promising improvement due to potential issues such as increased memory contention between concurrently executing threads and the extra branching overhead introduced by the nested loop structures.

```
@jit(nopython=True, cache=True)
def numba_tiled2_mul(A, B, C, B1_L2, B2_L1):
    N = A.shape[0]
    for i2 in range(0, N, B1_L2):
        for j2 in range(0, N, B1_L2):
            for k2 in range(0, N, B1_L2):
                for i1 in range(i2, min(i2 + B1_L2, N), B2_L1):
                    for j1 in range(j2, min(j2 + B1_L2, N), B2_L1):
                        for k1 in range(k2, min(k2 + B1_L2, N), B2_L1):
                            for i in range(i1, min(i1 + B2_L1, N)):
                                for j in range(j1, min(j1 + B2_L1, N)):
                                    tmp = 0.0
                                    for k in range(k1, min(k1 + B2_L1, N)):
                                        tmp += A[i, k] * B[k, j]
                                    C[i, j] += tmp
    return C
```

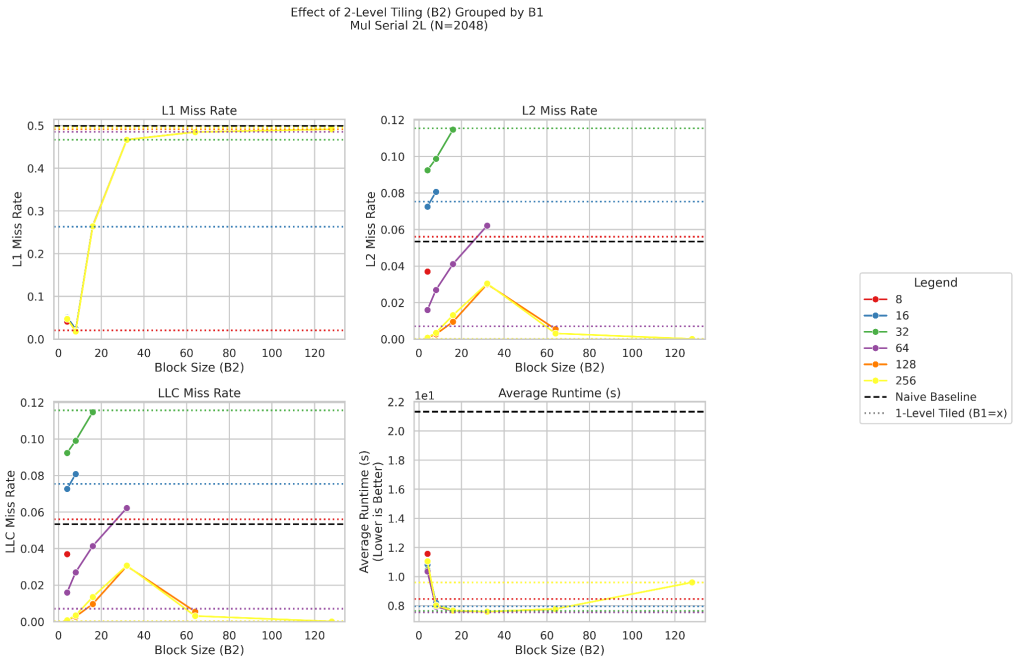


Fig 6: illustrates the influence of various 2nd level loop tiling block sizes on performance across different cache levels and the mean runtime. The naive benchmark is indicated in red dash, and each 1st layer loop blocking with different block size is also indicated in the corresponding color dash line in this graph.

3.4 Implementation of Loop Interchange in Numba

Loop interchange is a compiler optimization technique that reorders the nested loops in a program to improve memory access patterns **(Allen & Kennedy, 2002)**, specifically by enhancing spatial locality and temporal locality, which in turn reduces cache misses and speeds up execution. For matrix multiplication, *C = A @ B*, where *C[i][j] = sum(A[i][k] * B[k][j])*, there are 3! = 6 possible loop orderings (ijk, ikj, jik, jki, kij, kji), assuming *i* iterates over rows of *A* and *C*, *j* iterates over columns of *B* and *C*, and *k* iterates over the inner dimension. The baseline ordering provided, *ijk* (with *i* as the outermost loop), accesses *A* row-wise (*A[i][k]*) with good spatial locality, *B* column-wise (*B[k][j]*) with

poor spatial locality due to non-contiguous access, and *C* row-wise (*C[i][j]*) with good locality. Baseline benchmark is in order of ijk, where other code for each combination will be provided in the appendix.

The *ikj* loop order, which showed a performance speedup of *12.05x* over the *ijk* baseline in the non-parallel run, achieves this by significantly improving the access pattern for matrix *B*. In the *ikj* order, the loops are organized as *for i, for k, for j*. This structure causes *A[i][k]* to be accessed with good spatial and temporal locality, and *C[i][j]* is still accessed row-wise with good locality. This switch dramatically reduces cache misses accessing matrix B, making the *ikj* ordering one of the most cache-efficient loop orders for standard matrix multiplication without using a transposed *B*. Please note that if B transposed is used, the effect of loop interchange can be different. Due to the time limit for this project, we are not discussing the B transposed scenario.

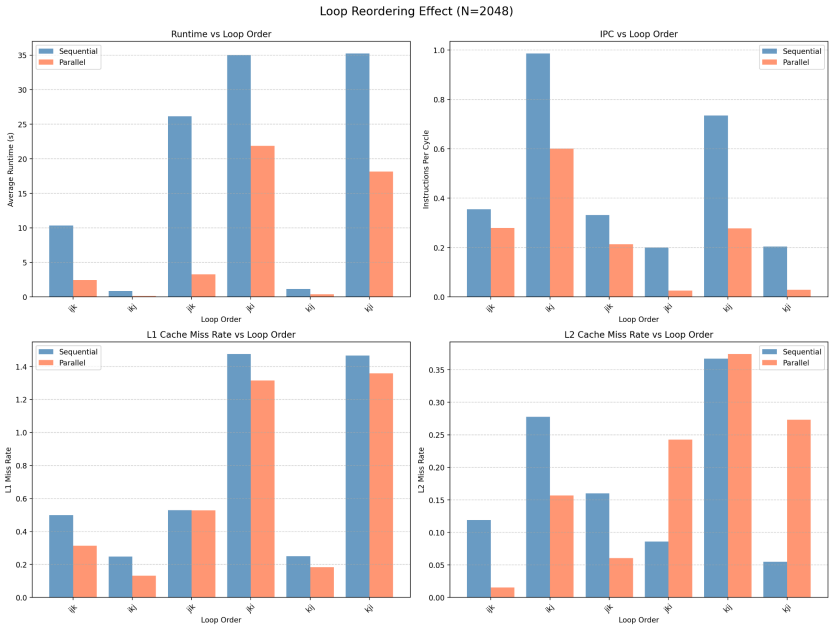


Fig 7: Impact of Loop Interchange on Execution Time and Cache Efficiency

3.5 Implementation of Loop Unrolling in Numba

Numba, by default, employs the Opt3 optimization level, which includes an unrolling pass in **LLVM (Lattner & Adve, 2004)** with a factor of 4. To accurately isolate the impact of the unrolling factor in our analysis, we will use `NUMBA_OPT=0` for this section. Since the absolute runtime with Opt0 is slower than with Opt3 (which is used in other benchmarks), we will also measure the naive implementation under Opt0 for a fair comparison within this specific test environment.

Loop unrolling is a crucial optimization technique that can significantly improve performance. It works by reducing the overhead associated with loop control, such as decrementing the loop counter and performing the jump back to the beginning of the loop. By expanding the loop body to execute multiple iterations in one go, it also creates opportunities for instruction-level parallelism, allowing the CPU to execute more operations concurrently, which can lead to better utilization of the processor's resources and ultimately, a faster runtime. Our unroll with a factor of 4 is implemented in:

```
@jit(nopython=True, cache=True)
def numba_unrolled4_mul(A, B, C):
    N = A.shape[0]
    for i in range(N):
```

```
            for j in range(N):
                tmp = 0.0
                for k in range(0, N - 3, 4):
                    tmp += A[i, k] * B[k, j]
                    tmp += A[i, k + 1] * B[k + 1, j]
                    tmp += A[i, k + 2] * B[k + 2, j]
                    tmp += A[i, k + 3] * B[k + 3, j]
                # Handle remainder
                for k in range((N // 4) * 4, N):
                    tmp += A[i, k] * B[k, j]
                C[i, j] = tmp
    return C
```

As shown in the graph below, unrolling with a factor of 4 yields the best runtime in the sequential case, providing a speedup of 2.06x. However, this optimization does not provide a noticeable benefit for the parallel execution case. This suggests that in the parallel context, the performance bottleneck or the optimization opportunities are dominated by other factors, such as thread management or memory access, rather than the loop overhead addressed by unrolling.

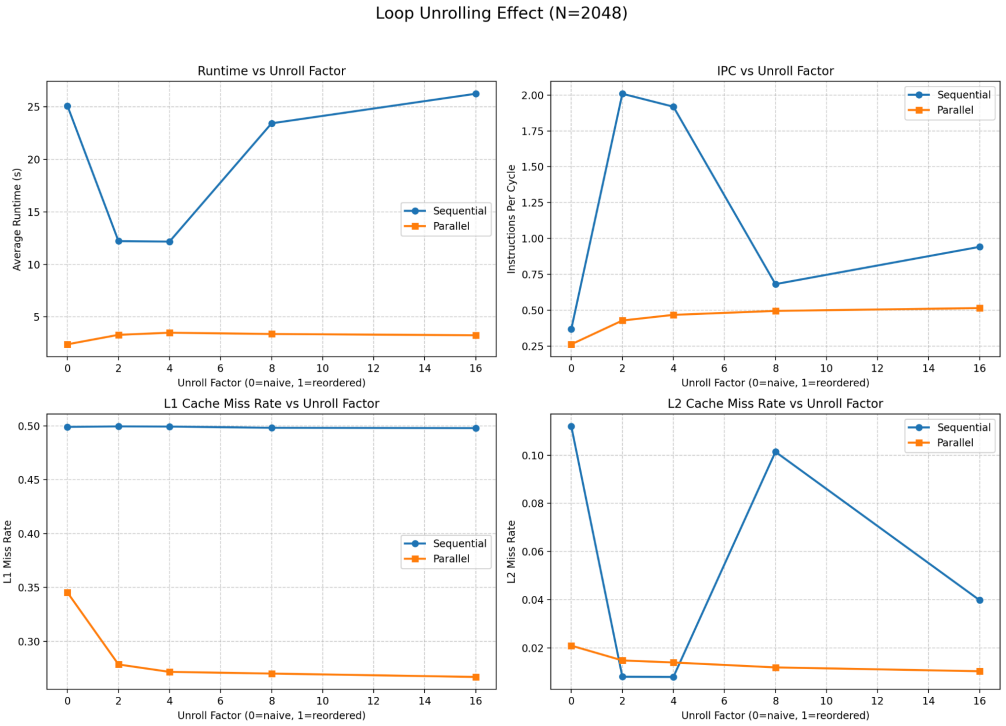


Fig 8: Impact of Loop Unrolling on Execution Time and Cache Efficiency

3.6 Combining All Numba Optimizations

By combining all the discussed techniques—loop tiling, using the transpose of B, loop interchange, loop unrolling (which is the default when using `NUMBA_OPT=3`), and parallelism—we created the ultimate benchmark, which is expected to run the fastest. The measurement will be further discussed in the next section.

```
@jit(nopython=True, cache=True, parallel=True)
def numba_ultimate_parallel(A, B_T, C, B1=32):
    N = A.shape[0]
    num_tiles = (N + B1 - 1) // B1
    for tile_idx_ii in prange(num_tiles):  # Parallelize over i tiles
```

```
            ii = tile_idx_ii * B1
            for tile_idx_kk in range(num_tiles):
                kk = tile_idx_kk * B1
                for tile_idx_jj in range(num_tiles):
                    jj = tile_idx_jj * B1
                    # ikj ordering within tiles
                    for i in range(ii, min(ii + B1, N)):
                        for k in range(kk, min(kk + B1, N)):
                            tmp = A[i, k]
                            for j in range(jj, min(jj + B1, N)):
                                C[i, j] += tmp * B_T[j, k]  # Transposed access
    return C
```

3.7 Performance Analysis and Discussion

From the overall analysis in the table below, we see that compared to a naive implementation of matrix multiplication, all techniques can improve performance. However, adding parallelism usually produces the most significant speedup. Without parallelism, loop interchange yields the best performance with a 12.06 speedup. Combining loop interchange with parallelism gives the overall best speedup of 82.82. Using B transpose combined with parallelism also provides a great speedup of 36.71. However, combining all techniques together does not yield the best overall performance. The possible reason for this is that the overhead introduced by combining multiple complex optimizations (like tiling) outweighs the benefits, especially when parallelism is already effectively mitigating the main bottleneck. This shows that the main bottleneck of matrix multiplication is the ability to concurrently run each operation in parallel.

Compared to the naive implementation *with* parallelism, Loop interchange + parallelism and B transpose + parallelism still give better speedups, 19.7 and 4.26, respectively. All techniques combined also do not yield the best performance relative to Naive + Parallel. This indicates that other than parallelism, simple techniques that utilize better cache locality, such as loop interchange, yield better performance. Tiling also utilizes locality but introduces extra branching overhead, which can sometimes negate its locality benefits.

Finally, compared to the highly-optimized NumPy `@` operation, none of these implementations achieve better performance. The reason for this is that NumPy's `@` operation is typically implemented using highly optimized, vendor-specific libraries like OpenBLAS or Intel MKL. These libraries are written in highly optimized C or Fortran and utilize assembly-level optimizations, extensive vectorization (SIMD), and sophisticated multi-threading and cache-aware techniques that are far more advanced and fine-tuned than the techniques explored here using Numba.

Speed up compare to Benchmark	Parallel	B Transpose	B Transpose + Parallel	Tiling (32)	Tiling (32) + Parallel	Loop Interchange (ikj)	Loop Interchange (ikj) + Parallel	Loop Unroll (4)	Loop Unroll (4) + Parallel	All	All + Parallel
Naive	8.64	2.94	36.71	2.78	17.93	12.06	82.82	2.06	7.18	3.43	19.47
Naive Parallel	1	0.34	4.26	0.32	2.08	4,16	19.7	0.197	0.683	0.398	2.23
Numpy @	0.007	0.003	0.035	0.003	0.017	0.024	0.163	0.001	0.006	0.003	0.0183

4. Triton Implementations and Results

The code of each implementation will be provided in the appendix.

4.1 Implementation of Loop Tiling in Triton

The GPU architecture naturally enforces tiling and blocking due to its hierarchical organization of processing units. The overall computation is divided into a grid of thread blocks, where each block is an independent unit of work assigned to a Streaming Multiprocessor (SM). This physical partitioning means that to manage the large number of threads and exploit on-chip shared memory for data reuse within a block, data must be loaded and processed in smaller, localized chunks, which is the essence of tiling.
Through empirical testing of various block sizes, the optimal block size for this matrix multiplication was determined to be m=64 (rows of C), n=64 (columns of C), and k=32 (columns of A).

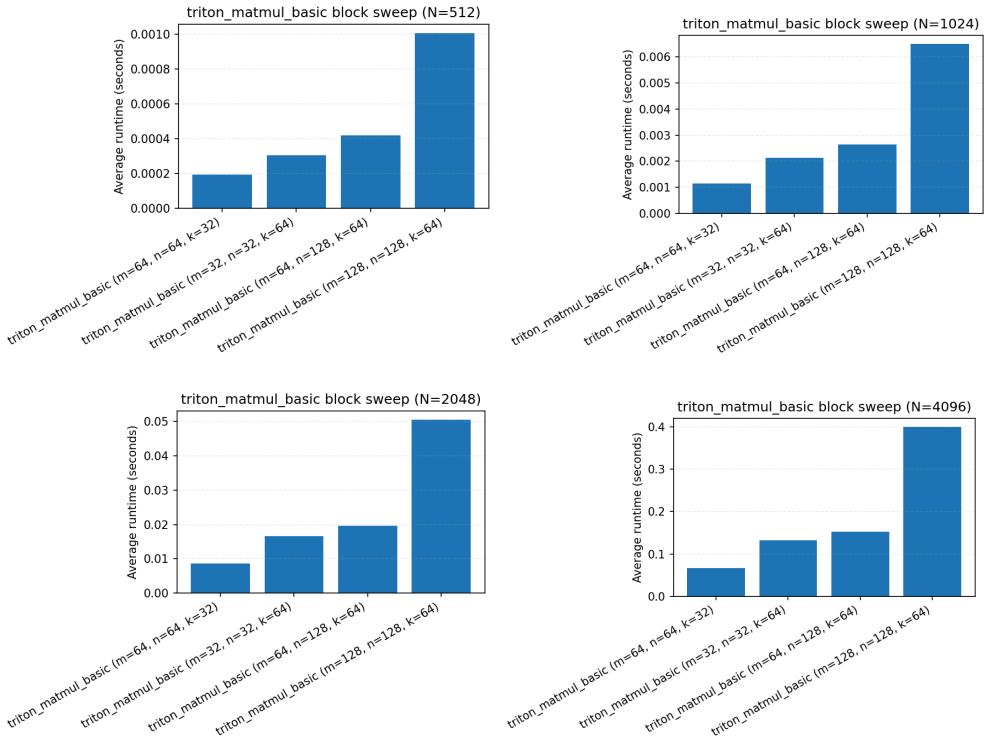


Fig 8: Impact of block size on Execution Time under different sizes of matrixes.

4.2 Implementation of 2D Grid Strategy

The 2D Grid strategy is a fundamental concept in CUDA programming **(NVIDIA, 2024)**, particularly effective for matrix operations. It involves structuring the execution space of a GPU kernel as a two-dimensional grid of thread blocks, where each thread block is identified by a unique `(blockIdx.x, blockIdx.y)` coordinate. In the context of matrix multiplication, this strategy typically maps each thread block to calculate a distinct tile (or sub-matrix) of the final output matrix C. This mapping allows for massively parallel computation because all blocks calculate their respective tiles concurrently. The 2D structure naturally aligns with the 2D nature of matrices, simplifying the coordination of threads and maximizing the utilization of the GPU's thousands of parallel processing cores, thus offering a significant performance improvement over a 1D grid strategy.

In triton, 2D grid is done by `pid_m=tl.program_id(axis=0)` and `pid_n=l.program_id(axis=1)`, rather than in the basic kernel is was implemented by:

```
# naive triton kernel implementation
pid = tl.program_id(axis=0)
num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
pid_m = pid // num_pid_n
pid_n = pid % num_pid_n
```

4.3 Implementation of L2 Swizzling

Standard row-major execution order can lead to "cache thrashing" in the L2 cache. If thread blocks proceed linearly row-by-row, the data loaded for matrix B may be evicted from the L2 cache before it is needed by the next row of blocks. L2 Swizzling reorders the execution path to process a "super-group" of rows (e.g., 8 rows) together before moving to the next set of columns. This may significantly increase the reuse of matrix B tiles in the L2 cache.
We implemented this introducing a GROUP_M constant (set to 8 in our configurations). We replaced the standard linear PID mapping with a swizzled mapping logic. The kernel calculates a group_id and iterates through a dense rectangular block of the output matrix defined by GROUP_M times the grid width.

```
pid = tl.program_id(axis=0)

num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
num_pid_in_group = GROUP_M * num_pid_n
group_id = pid // num_pid_in_group
first_pid_m = group_id * GROUP_M
group_size = min(num_pid_m - first_pid_m, GROUP_M)

pid_m = first_pid_m + (pid % group_size)
pid_n = (pid % num_pid_in_group) // group_size
```

The utilization of this swizzled mapping logic, while advantageous for L2 cache reuse, introduces increased overhead in terms of instruction execution and register allocation compared to a straightforward linear mapping. The computation of group_id, first_pid_m, and the subsequent derivation of pid_m and pid_n from the linear pid necessitates a greater number of arithmetic operations and potentially higher register pressure to store intermediate values. Furthermore, it may disrupt the favorable locality of access for matrix A compared to a naive implementation. Consequently, this complexity warrants a careful performance analysis to determine if the benefits in cache utilization outweigh the associated computational and resource costs.

4.4 Implementation of Persistent Kernels

Persistent kernels **(Gupta et al., 2012)** are a technique used to improve the performance of GPU computations, particularly in scenarios where the overhead of launching and scheduling individual kernel executions becomes a bottleneck. In a conventional GPU execution model, each parallel task (like a matrix multiplication) requires a separate kernel launch. This incurs scheduling overhead at the host (CPU) and device (GPU) level for every single execution. By contrast, a persistent kernel launches a single, long-running kernel that stays active for the entire duration of the computational task. The work partitioning and scheduling across the available Streaming Multiprocessors (SMs)—such as the 22 SMs on a GTX 1660—are managed internally within this single kernel. This approach effectively amortizes the initial launch overhead over the entire workload, leading to a performance improvement by reducing the time spent on administrative tasks.

However, moving the scheduling logic from the GPU driver and runtime system into the kernel code itself introduces a different kind of overhead: too many persistent threads try to access the single work queue at the same time, creating a bottleneck that the hardware scheduler handles better. A persistent kernel only brings benefits when the scheduling execution is costly.

In the Triton framework, this persistent kernel approach is implemented using a global loop that ensures the launched kernel remains active and continuously schedules matrix multiplication blocks. The provided code snippet illustrates how this internal scheduling is managed. Each program instance (`pid`) starts with a tile ID (`start_pid`) and steps by the number of available SMs (`NUM_SMS`) to ensure all hardware resources are utilized efficiently.

```
start_pid = tl.program_id(axis=0)
num_pid_m = tl.cdiv(M, BLOCK_SIZE_M)
num_pid_n = tl.cdiv(N, BLOCK_SIZE_N)
total_tiles = num_pid_m * num_pid_n

# Each program instance steps by the grid size to process more tiles
for pid in range(start_pid, total_tiles, NUM_SMS):
    num_pid_in_group = TILE_GROUP_SIZE * num_pid_n
    group_id = pid // num_pid_in_group
    first_pid_m = group_id * TILE_GROUP_SIZE
    group_size_m = min(num_pid_m - first_pid_m, TILE_GROUP_SIZE)

    pid_m = first_pid_m + (pid % group_size_m)
    pid_n = (pid % num_pid_in_group) // group_size_m
   …
```

4.5 Autotune in Triton

Triton's kernel performance is highly sensitive to configuration parameters, such as block sizes for the matrix multiplication, which define the granularity of parallel work. To achieve optimal performance, Triton provides an `autotune` feature that systematically explores a range of these parameters, searching for the most efficient setup for a given hardware and matrix size. For instance, it can determine the ideal block sizes of the grid for running the matrix multiplication kernel. However, while essential for maximizing speed, this `autotune` process introduces a non-trivial overhead due to the need for "warm-up" runs and the total time spent iterating through all potential configurations. This overhead means that for applications where the kernel is run infrequently or for very small matrices, the benefit of autotuning might be offset by its initial cost.

Initially, I intended to implement a prefetching/pipelining kernel manually; however, due to time constraints and the excellent features provided by Triton, I was able to configure pipelining and prefetching using the tool's built-in parameters. The `num_warps` parameter dictates how many GPU warps (groups of 32 threads) execute the kernel concurrently, directly impacting parallelism. The `num_stages` parameter, in conjunction with the technique of *software prefetching*, determines the memory pipeline depth. Prefetching is crucial because it allows the GPU to initiate the loading of the *next* required data blocks from memory while the current data blocks are still being processed. This overlap of computation and memory access effectively hides memory latency, which is often a major bottleneck in matrix multiplication. A well-tuned `num_stages` ensures that the data is ready just in time, maximizing GPU core utilization.

To see the detailed autotune results, you typically need to set the `TRITON_PRINT_AUTOTUNING=1` environment variable before running the kernel. The one of the best config I tried in my script for running N=2048 on GTX1660 is BLOCK_SIZE_M: 64, BLOCK_SIZE_N: 64, BLOCK_SIZE_K: 32, num_warps: 4, num_stages: 3, num_warps: 4, num_ctas: 1, num_stages: 3, which matched the result in our manual measurement of the basic benchmark in section 4.1.

4.6 Performance Analysis and Discussion

Comparing each technique, the basic benchmark or autotuned basic benchmark always yields the best performance among all Triton kernels tested. The other techniques, such as the persistent kernel, 2D grid, and L2 swizzling, offered very limited performance improvement and, most of the time, introduced overhead in our matrix multiplication case. This suggests that the bottleneck may not be in scheduling but in another area, such as the memory wall, which means the processor spends significant time waiting for the DRAM to bring in the data.

Our best Triton kernel can sometimes outperform the CuPy library, but the difference is minimal. CuPy optimizes matrix multiplication using techniques like highly-tuned vendor libraries (e.g., cuBLAS), advanced memory management, and kernel fusion.

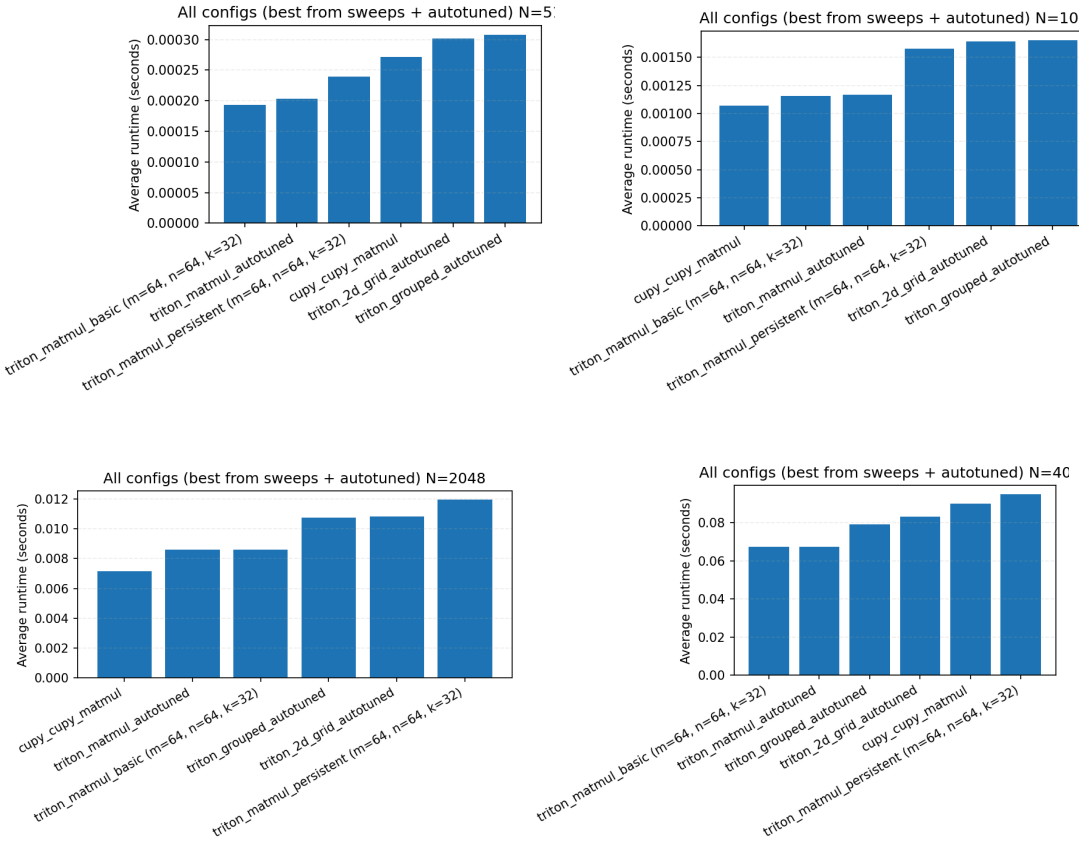


Fig 9: Impact of different techniques/library CUDA kernel implementation on Execution Time under different sizes of matrixes.

5. Comparative Analysis

Due to time constraints, it was not feasible to execute a pure Python naive matrix multiplication at the same scale as the other experiments; however, a single pure Python execution of one 512x512

matrix multiplication still required 22.41 seconds to complete. Just by employing the Numba JIT compiler on naive code, we were able to perform 30 executions of 2048x2048 matrix multiplication in 21.22 seconds, which represents a significant improvement. Further optimization through CPU parallelism, utilizing threading and loop interchange, reduced the time for 30 executions of 2048x2048 matrix multiplication to 0.125 seconds. To achieve even greater parallelism, the use of Triton to write a CUDA kernel for the GPU allowed the same scaled operation to run in 0.0065 seconds, even with only blocking implemented and without other optimizations in the CUDA kernel. This trajectory demonstrates the combined impact of software development techniques (compiler optimization and parallelism algorithms) and hardware utilization (leveraging CPU cores and GPU capabilities) on computational performance.

6. Conclusion and Future Work

6.1 Summary of Findings

This research quantitatively evaluated the efficacy of Just-In-Time (JIT) compilation techniques in Python for matrix multiplication, specifically contrasting CPU optimizations via Numba with GPU optimizations via Triton. The experimental results lead to several key conclusions regarding the performance hierarchy and the effectiveness of specific optimization strategies.

CPU Optimization (Numba):

- **Parallelism is Paramount:** The investigation demonstrated that while algorithmic optimizations like loop interchange and tiling provide tangible benefits in sequential execution, explicit parallelization (threading) yields the most significant performance leap.
- **Algorithmic Efficiency:** Among loop transformations, Loop Interchange (ikj ordering) proved to be the most effective single optimization, leveraging spatial locality to achieve a massive speedup when combined with parallelism (82.82x over naive).
- **Diminishing Returns on Combining Optimization:** While blocking (tiling) is theoretically sound for cache management, the experiments revealed that combining complex tiling logic with multi-threading and transposition can lead to cache contention or scheduling overheads that negated the benefits.

GPU Optimization (Triton):

- **Hardware Superiority:** The transition from CPU to GPU execution using Triton resulted in orders-of-magnitude performance gains, with execution times dropping from hundreds of milliseconds (CPU) to mere milliseconds (GPU).
- **Simplicity Wins:** Contrary to initial expectations, complex scheduling techniques such as **Grouped GEMM** and **Persistent Kernels** did not outperform the baseline tiled Triton kernel on the tested hardware (GTX 1660). The introduced instruction overhead and register pressure outweighed the benefits for the tested matrix size (2048x2048).

6.2 Limitations

While this study provides valuable insights into JIT optimization, several limitations characterize the scope and results:

1. **Hardware Constraints:** The experiments were conducted on a single consumer-grade GPU (GTX 1660, Turing architecture) and a specific CPU (Intel i7-11700). The benefits of

advanced Triton features, such as Persistent Kernels, might only become apparent on enterprise-grade hardware (e.g., NVIDIA A100 or H100) with higher SM counts and memory bandwidth.

2. **Fixed Problem Size:** The benchmarks focused exclusively on square matrices of size 2048x2048. This specific dimension may not expose the scalability issues or overheads that arise with very small matrices (where launch latency dominates) or extremely large matrices (where memory capacity becomes the bottleneck).
3. **Data Precision:** All experiments used single-precision floating-point (FP32). Modern machine learning workloads increasingly rely on half-precision (FP16) or **Brain Floating Point (BF16) (Wang & Kanwar, 2019)** to leverage Tensor Cores **(NVIDIA, 2018)**, which were not utilized in this study.
4. **Tuning Overhead:** The Triton autotuning process, while effective, introduces a warm-up cost that was not factored into the primary execution time comparisons. In dynamic production environments, this overhead could be non-trivial.

6.3 Recommendations for Further Research

To build upon these findings and address the identified limitations, future research should focus on the following areas:

- **Tensor Core Utilization:** Extending the Triton implementation to utilize FP16/BF16 data types would allow the kernel to leverage the Tensor Cores available on modern NVIDIA GPUs. This is essential for benchmarking against state-of-the-art Deep Learning frameworks.
- **Kernel Fusion:** A major advantage of Triton is the ability to fuse operations (e.g., Matrix Multiplication followed by Activation). Future work should benchmark "fused" kernels against standard library chains (e.g., `torch.matmul` + `torch.relu`) to quantify the memory bandwidth savings.
- **Architecture-Aware Tuning:** The study should be replicated across different GPU architectures (e.g., Ampere, Hopper) to determine if the complex scheduling techniques (Swizzling/Persistence) yield better results on hardware with different cache hierarchies and SM counts.
- **SIMD Explicit Vectorization:** For Numba, further research could explore explicit SIMD vectorization directives or the use of `llvmlite` to bridge the gap between the Numba generated code and the AVX-512 optimized routines found in OpenBLAS.

7. Appendix

7.1 Full Code Listing

https://github.com/laiy24/matmul_numba_triton

8. Reference

Allen, R., & Kennedy, K. (2002). Optimizing compilers for modern architectures: A dependence-based approach. Morgan Kaufmann.

Gupta, K., Stuart, J. A., & Owens, J. D. (2012, May). A study of persistent threads style GPU programming for GPGPU workloads. In Innovative Parallel Computing (InPar '12) (pp. 1-14). IEEE. https://doi.org/10.1109/InPar.2012.6339601

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... & Oliphant, T. E. (2020). Array programming with NumPy. Nature, 585(7825), 357-362. https://doi.org/10.1038/s41586-020-2649-2

Lam, S. K., Pitrou, A., & Seibert, S. (2015, November). Numba: A LLVM-based Python JIT compiler. In Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (pp. 1-6). ACM. https://doi.org/10.1145/2833157.2833162

Lattner, C., & Adve, V. (2004). LLVM: A compilation framework for lifelong program analysis & transformation. In Proceedings of the international symposium on Code generation and optimization: feedback-directed and runtime optimization (p. 75). IEEE Computer Society.

NVIDIA Corporation. (2018). NVIDIA Turing Architecture Whitepaper. Retrieved from https://images.nvidia.com/aem-dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf

NVIDIA Corporation. (2024). CUDA C++ Programming Guide. Retrieved from https://docs.nvidia.com/cuda/cuda-c-programming-guide/

Okuta, R., Unno, Y., Nishino, D., Hido, S., & Loomis, C. (2017, June). CuPy: A NumPy-compatible library for NVIDIA GPU calculations. In Proceedings of Workshop on Machine Learning Systems (LearningSys) in The Thirty-first Annual Conference on Neural Information Processing Systems (NIPS).

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*, 8024–8035. http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf

Tillet, P., Kung, H. T., & Cox, D. (2019, June). Triton: an intermediate language and compiler for tiled neural network computations. In Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (pp. 10-19). ACM. https://doi.org/10.1145/3315508.3329973

Zhang, X., Wang, Q., & Zhang, Y. (2012). Model-driven Level 3 BLAS performance optimization on Loongson 3A processor. *2012 IEEE 18th International Conference on Parallel and Distributed Systems (ICPADS)*, 684–691. https://doi.org/10.1109/ICPADS.2012.97

Wang, S., & Kanwar, P. (2019). BFloat16: The secret to high performance on Cloud TPUs. Google Cloud Blog. Retrieved from https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus

Wolf, M. E., & Lam, M. S. (1991). The cache performance and optimizations of blocked algorithms. ACM SIGOPS Operating Systems Review, 25(Special Issue), 63-74. https://doi.org/10.1145/106973.106981